

LV04 Direct기법, 2중 포인터, 2차원 배열 패턴찾기

■ 소유자	 암암코딩
■ 태그	
■ 생성 일시	@2023년 12월 16일 오후 7:09

<https://youtu.be/NyH7ogsi3yo?feature=shared>

C++ 방향 배열, 포인터, 구조체 활용

방향을 표현하기 위한 코딩 방법 (Direct 기법)

방향 배열의 활용

스타듀밸리 같은 2D 게임에서 낚시줄 할 때 기준된 주변의 8칸을 하고, 콤보리게 할 때 앞에 3칸을 탐색하는 것처럼 로직을 각각 정해두고 사용할 수 있다.

젤다의 전설같은 3D 게임도 사실 2차원 평면도에 따라 로직을 구성한 것이다.

Direct 방향 배열 구현

```
#include <iostream>
using namespace std;

int main()
{
    int arr[3][3] =
    {
        1, 2, 3,
        4, 5, 6,
        7, 8, 9
    };
};
```

```

// 기준점을 (0,0) 으로 뒀을 때 좌표값의 배열
int direct[4][2] =
{
    -1, 0, // 상
    1, 0, // 하
    0, -1, // 좌
    0, 1 // 우
};

int y = 1;
int x = 1;

// 입력받을 경우
// int x, y;
// cin >> y >> x;

int sum = 0;
// 4X2 배열 direct 에서
for (size_t i = 0; i < 4; i++)
{
    int newY = y + direct[i][0];
    int newX = x + direct[i][1];
    // ex) newX = 기준의 좌표값에 direct 배열의 x 좌표값을 더한다.
    // ex) newY = 1 + direct[0][0] (= 1) ⇒ 0
    // ex) newX = 1 + direct[0][1] (=0) → 1
    // ex) newY, newX = 0, 1
    // ex) arr[0][1] = 2

    if (newX >= 0 && newX <= 2 && newY >= 0 && newY <= 2)
    {
        sum += arr[newY][newX];
    }
}

cout << sum << endl;

return 0;
}

```

동작 원리:

1. **기준점 설정:** (1, 1) 위치에서 시작
2. **방향 배열:** 상(-1,0), 하(1,0), 좌(0,-1), 우(0,1)
3. **좌표 계산:** 기준점 + 방향 벡터 = 새로운 좌표
4. **경계 검사:** 배열 범위를 벗어나지 않는지 확인
5. **값 누적:** 유효한 좌표의 값들을 합산

실행 결과: 기준점 (1,1)의 상하좌우 값들의 합 = 2 + 8 + 4 + 6 = 20

8방향 탐색 확장

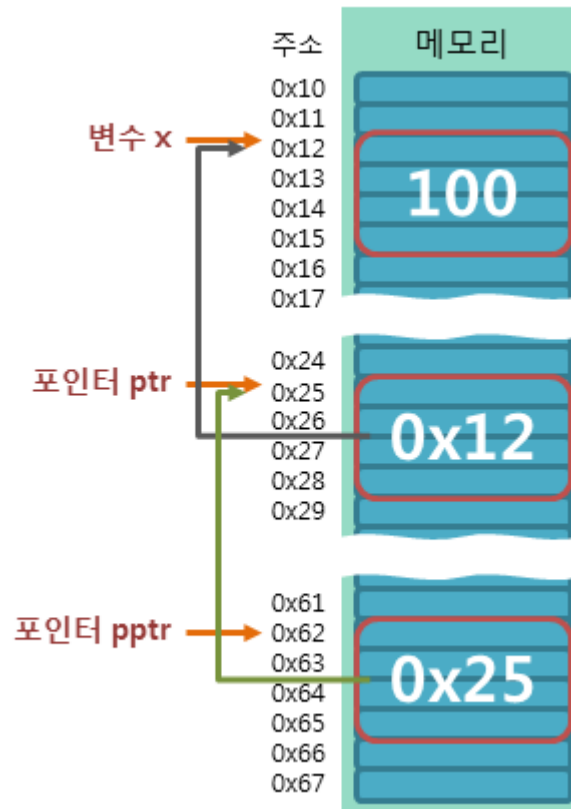
```
int direct8[8][2] =
{
    -1, -1, // 좌상
    -1, 0, // 상
    -1, 1, // 우상
    0, -1, // 좌
    0, 1, // 우
    1, -1, // 좌하
    1, 0, // 하
    1, 1 // 우하
};
```

2중 포인터 변수

포인터의 포인터 개념

2중 포인터는 포인터 변수의 주소를 저장하는 포인터입니다. 메모리 구조를 이해하는 것이 핵심입니다.

메모리 구조 시각화



메모리 주소	값	변수명
0x12	100	num (변수)
0x25	0x12	ptr (포인터)
0x61	0x25	pptr (포인터의 포인터)

2중 포인터 기본 사용법

```
#include <iostream>
using namespace std;

// 포인터 변수 = 주소값(주소 타입)을 저장하는 변수 타입
// int* P = &a; (a의 주소값을 포인터 변수의 값으로 가진다)
// int 변수 = 정수형 숫자를 저장하는 변수 타입
// char 변수 = 문자를 글자를 저장하는 변수 타입

// 2중 포인터 변수
// (int a = 100) == (int* P = &a) == (int** PP = &P) == (&P = 0X17)
```

```

int main()
{
    int a = 100;
    int* p = &a;
    int** pp = &p;

    *(*pp) = 200;

    // 결과 a의 값이 변한다.

    return 0;
}

```

2중 포인터 사용 과정:

1. `int a = 100` : 정수 변수 a에 100 저장
2. `int* p = &a` : 포인터 p가 a의 주소 저장
3. `int** pp = &p` : 포인터의 포인터 pp가 p의 주소 저장
4. `(*pp) = 200` : pp를 통해 a의 값을 200으로 변경

접근 과정:

- `pp` : p의 값 (a의 주소)
- `(*pp)` : a의 값
- `(*pp) = 200` : a의 값을 200으로 변경

2중 포인터의 실용적 활용

```

#include <iostream>
using namespace std;

void ChangeValue(int** ptr)
{
    **ptr = 500; // 원본 값 변경
}

void ChangePointer(int** ptr, int* newTarget)
{

```

```

    *ptr = newTarget; // 포인터가 가리키는 대상 변경
}

int main()
{
    int a = 100;
    int b = 300;
    int* p = &a;

    cout << "초기값: *p = " << *p << endl; // 100

    ChangeValue(&p);
    cout << "값 변경 후: *p = " << *p << endl; // 500

    ChangePointer(&p, &b);
    cout << "포인터 변경 후: *p = " << *p << endl; // 300

    return 0;
}

```

구조체, 클래스를 이용한 배열

구조체를 이용한 데이터 관리

```

#include <iostream>
using namespace std;

// 구조체, 클래스를 이용한 배열
struct ABC
{
public:
    // 생성자 (해당사항 없음)

    // 소멸자 (해당사항 없음)

    // 멤버 함수들 (해당사항 없음)

private:

```

```

// 멤버 변수들
int mA;
int mB;
};

int main()
{
    ABC arr[3];

    arr[0].mA = 100;
    arr[0].mB = 200;

    cin >> arr[0].mA >> arr[0].mB;
    cout << arr[0].mA << arr[0].mB;

    return 0;
}

```

주의: 위 예제에서는 구조체의 멤버 변수가 private으로 되어 있어 외부에서 직접 접근할 수 없습니다. 실제로는 public으로 선언하거나 getter/setter 함수를 제공해야 합니다.

올바른 구조체 구현

```

struct ABC
{
public:
    int mA;
    int mB;

    // 생성자
    ABC(int a = 0, int b = 0) : mA(a), mB(b) {}

    // 입력 함수
    void Input()
    {
        cin >> mA >> mB;
    }
}

```

```
// 출력 함수
void Output()
{
    cout << mA << " " << mB << endl;
}
};
```

패턴찾기 → 2중 for문에서 패턴찾기

2차원 배열에서 패턴 매칭

2차원 배열에서 특정 패턴을 찾는 알고리즘입니다.

```
#include <iostream>
using namespace std;

// 패턴찾기 -> 2중 For문에서 패턴찾기

int map[5][5] =
{
    1, 3, 5, 1, 5,
    3, 7, 9, 5, 8,
    4, 5, 3, 9, 7,
    2, 7, 6, 1, 9,
    1, 6, 2, 3, 8
};

int pattern[2][2] =
{
    3, 5,
    7, 9
};

int IsPattern(int dy, int dx)
{
    for (size_t y = 0; y < 2; y++)
    {
        for (size_t x = 0; x < 2; x++)
```



```

    {
        if (map[dy + y][dx + x] != pattern[y][x])
        {
            return 0;
        }
    }
}
return 1;
}

int main()
{
    for (size_t y = 0; y < 4; y++)
    {
        for (size_t x = 0; x < 4; x++)
        {
            int result = IsPattern(y, x);
            if (result == 1)
            {
                cout << "존재" << endl;
                return 0;
            }
        }
    }

    cout << "노존재" << endl;

    return 0;
}

```

패턴 매칭 과정:

1. 시작 위치 설정: (dy, dx)에서 패턴 검사 시작
2. 패턴 크기만큼 반복: 2x2 패턴이면 4번 비교
3. 일치 여부 확인: 모든 위치의 값이 일치하는지 검사
4. 결과 반환: 일치하면 1, 불일치하면 0 반환

문자 패턴 찾기

```

#include <iostream>
using namespace std;

char map[3][4] =
{
    {'A', 'B', 'G', 'K'},
    {'T', 'T', 'A', 'B'},
    {'A', 'C', 'C', 'D'}
};

char pattern[2][2];

char IsPattern(char y, char x)
{
    for (size_t y = 0; y < 2; y++)
    {
        for (size_t x = 0; x < 2; x++)
        {
            if (map[y][x] != pattern[y][x])
            {
                return 0;
            }
        }
    }
    return 1;
}

char main()
{
    for (size_t i = 0; i < 2; i++)
    {
        cin >> pattern[i][0];
        cin >> pattern[i][1];
    }

    for (size_t i = 0; i < 2; i++)
    {
        for (size_t j = 0; j < 3; j++)

```

```

    {
        int result = IsPattern(i, j);
        if (result == 1)
        {
            cout << "발견\n";
        }
        else
        {
            cout << "미발견\n";
        }
    }
}

return 0;
}

```

실무에서의 활용

게임 개발에서의 방향 배열

RPG 게임에서의 활용:

```

enum Direction
{
    UP    = 0,
    DOWN  = 1,
    LEFT  = 2,
    RIGHT = 3
};

class Player
{
public:
    Player(int y, int x) : mY(y), mX(x) {}

    void Move(Direction dir)
    {
        int newY = mY + mDirections[dir][0];

```

```

    int newX = mX + mDirections[dir][1];

    if (IsValidPosition(newY, newX))
    {
        mY = newY;
        mX = newX;
    }
}

bool IsValidPosition(int y, int x)
{
    return (y >= 0 && y < MAP_HEIGHT && x >= 0 && x < MAP_WIDTH);
}

private:
    static const int mDirections[4][2];
    int mY, mX;
    static const int MAP_HEIGHT = 10;
    static const int MAP_WIDTH = 10;
};

const int Player::mDirections[4][2] =
{
    {-1, 0}, // UP
    { 1, 0}, // DOWN
    { 0, -1}, // LEFT
    { 0, 1}  // RIGHT
};

```

패턴 매칭의 실무 활용

이미지 처리에서의 템플릿 매칭:

- 특정 패턴의 객체 찾기
- 얼굴 인식, 문자 인식
- 게임에서 아이템 배치 패턴 인식

데이터 마이닝:

- 시계열 데이터에서 특정 패턴 찾기
- 로그 분석에서 이상 패턴 탐지
- 바이오인포매틱스에서 DNA 서열 분석

메모리 효율성 고려사항

2중 포인터 사용 시 주의점:

1. **메모리 누수 방지:** 동적 할당 시 반드시 해제
2. **NULL 포인터 체크:** 포인터 사용 전 유효성 검사
3. **깊은 복사 vs 얇은 복사:** 포인터 복사 시 주의

방향 배열과 패턴 매칭은 **게임 개발, 이미지 처리, 데이터 분석** 등 다양한 분야에서 핵심적으로 활용되는 기법입니다. 특히 **2D 게임의 맵 탐색**이나 **이미지 인식 알고리즘**에서 필수적인 도구로 사용됩니다.

"강의는 많은데, 왜 나는 아직도 코드를 못 짤까?"

혼자 공부하다 보면 누구나 이런 고민을 하게 됩니다.

- 강의는 다 들었지만 막상 **손이 안 움직이고**,
- 복습을 하려 해도 **무엇을 다시 봐야 할지 모르겠고**,
- 질문할 곳도 없고,
- 유튜브는 결국 **정답을 따라 치는 것밖에 안 되는 것 같고**.

문제는 '**연습**'이 빠졌기 때문입니다.

단순히 강의를 듣는 것만으로는 실력이 늘지 않습니다.

실제 문제를 풀고, 고민하고, 직접 구현해보는 시간이 반드시 필요합니다.

그래서, 얄얄코딩 코칭은 다릅니다.

그냥 가르치지 않습니다.

스스로 설계하고, 코딩할 수 있게 만듭니다.

얄얄코딩 코칭에서는 단순한 예제가 아닌,

스스로 문제를 분석하고 구현해야 하는 연습문제를 제공합니다.

이 연습문제들은 다음과 같은 역량을 키우기 위해 설계되어 있습니다:

- 문제를 스스로 쪼개고 설계하는 힘
- 다양한 조건을 만족시키는 실제 구현 능력
- 기능 단위가 아닌, 프로그램 단위로 사고하는 습관
- 마침내 자신의 힘으로 코드를 끝까지 작성하는 경험

지금 필요한 건 더 많은 강의가 아닙니다.

코드를 스스로 완성해 나가는 훈련,

그것이 지금 실력을 끌어올릴 가장 현실적인 방법입니다.

👉 자세한 안내 보기: [프리미엄 코칭 안내 바로가기](#)

👉 또는 카카오톡 상담방: [얌얌코딩 상담방](#)

▼ oopy:hide

연습문제



문제 1번

3	5	4
1	1	2
1	3	9

위 3x3 배열을 **하드코딩** 해주세요.

그리고 **좌표 하나를 입력** 받아주세요. (y, x)

입력받은 좌표에서 **바로 윗칸, 아랫칸, 왼쪽칸, 오른쪽칸의 합을**

Direct 방식을 이용한 함수를 만들어 주세요.

그리고 그 결과를 출력 해주세요.

ex1) **11** 입력시 (y, x)

	5	
1		2
	3	

이므로 $5+1+2+3 = 11$ 출력

ex2) **01** 입력시 (y, x)

3		4
	1	

유효한 칸은 세칸이므로 $3+1+4 = 8$ 출력

입력 예제

11

출력 결과



문제 2번

Data	
int x	
int y	
int z	

Data 구조체를 만들고 구조체 변수 **a**, **b**를 만들어 주세요.

숫자 6개를 **a.x**, **a.y**, **a.z**, **b.x**, **b.y**, **b.z** 에 각각 **입력** 받고,

a.x + b.x

a.y + b.y

a.z + b.z 값을 출력 해주세요.

ex)

입력:

1 2 3

4 5 6

출력:

5

7

9

입력 예제

1 2 3

4 5 6

출력 결과

5

7

9





문제 3번

MC

burger1								
burger2								

왼쪽과 같은 형태의 구조체를 정의해주세요.

그리고 구조체 변수 bob, tom을 만들고, 문장 4개를 구조체 변수에 입력 받아주세요.



sanghi, bulgogi, chicken, dove

만약 네 문장을 아래와 같이 입력 받았다면, 아래와 같이 채워주세요.

bob		tom	
burger1	s a n g h i	burger1	c h i c k e n
burger2	b u l g o g i	burger2	d o v e

이제 각각 몇글자인지 출력하면 됩니다.

입력 예제

sanghi

bulgogi

chicken

dove

출력 결과

bob.burger1=6

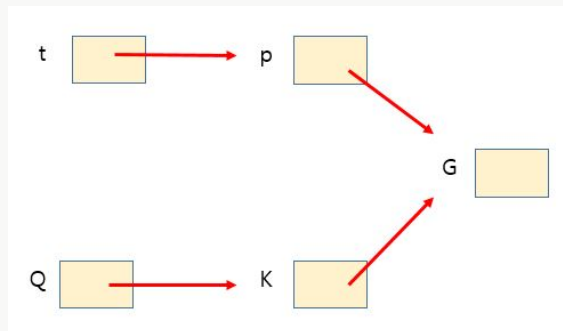
bob.burger2=7

tom.burger1=7

tom.burger2=4



문제 4번



t와 Q는 더블포인터 입니다.

위 상태를 구현하고

변수 G에다가 숫자 1개를 입력 받으세요.

그리고 ****t, *K**의 값을 출력 해주세요.

입력 예제

5

출력 결과

5 5



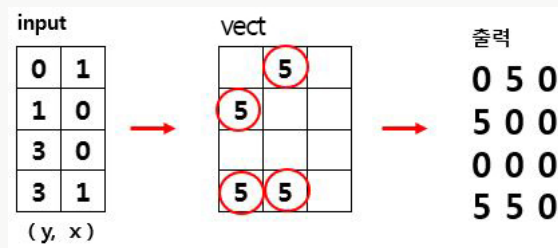
문제 5번

네쌍의 좌표를 input 배열에 입력 받아주세요.

그리고 vect배열 4x3에 배열을 만들고 0으로 초기화 해 주세요.

vect배열에서 input 배열에 든 좌표에 해당하는 곳을 찾아 그 좌표에 숫자 5를 넣어주세요.

그리고 그 결과를 출력 해주세요.



[HINT]

```
vect[0][1] = 5;
```

```
vect[1][0] = 5;
```

```
vect[3][0] = 5;
```

```
vect[3][1] = 5;
```

입력 예제

0 1

1 0

3 0

3 1

출력 결과

0 5 0

5 0 0

0 0 0

5 5 0



문제 6번

	0	1	2	3	4	5	6
	win 15 name summer	win 33 name cloe	win 24 name summer	win 28 name niki	win 32 name jenny	win 20 name summer	win 40 name coco

위와 같이 구조체 기차에 사람들이 타고 있습니다.

win에는 나이가

name에는 이름이 적어져 있습니다.

tom은 첫사랑을 찾기 위해 이 기차를 탔습니다. (구조체 배열 기차 입니다)

tom의 첫사랑 이름과 나이를 입력받고, 몇번 index에 있는지 찾아서 출력해주세요.

[힌트1] 구조체변수 하드코딩 하기

```
#include<iostream>

using namespace std;

struct Node

{

    int x, y;

};

int main()

{

    Node a = { 3, 4 };

    Node b = { 5, 6 };

    cout << a.x << " " << a.y << endl;
```

```
cout << b.x << " " << b.y << endl;

}
```

[힌트2] 구조체배열 하드코딩하기

```
#include<iostream>

using namespace std;

struct Train

{

int win;

char name[8];

};

int main()

{

Train t[3] = { {35, "ABC"}, {100, "BBQ"}, {15, "KFC"} };

cout << t[0].name;

}
```

[힌트3] 문장은 비교가 되지 않습니다.

```
char a[10] = "BBQ";
char b[20] = "ABC";
if (a == b) //error!
```

for문을 돌려 한 글자씩 비교를 해주어야 합니다.

함수를 만들어 비교를 하면 됩니다.

입력 예제

summer

20

출력 결과

5



문제 7번

map

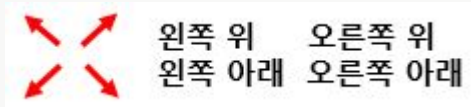
3	3	5	3	1
2	2	4	2	6
4	9	2	3	4
1	1	1	1	1
3	3	5	9	2

map 배열을 하드코딩 하주세요.

그리고 **sum(y,x)** 함수를 만들어 주세요.

이 함수는 특정좌표(y,x)에서

왼쪽위, 오른쪽 위, 왼쪽 아래, 오른쪽 아래의 합을 return 해주는 함수입니다.



이 sum함수를 이용해서 **가장 큰 값이 나오는 좌표를 출력**하세요.

(direct 기법을 사용해주세요, 입력값은 없습니다.)

출력 결과

3 2



문제 8번

-	-	-	-	-
-	-	-	-	-
-	-	-	-	-
-	-	-	-	-

4×5 char 배열을 준비해주세요.

값을 '-' 문자로 꼭 채워주세요.

폭탄을 투하할 좌표 두곳을 입력 받아주세요.(y,x)

만약 폭탄이 (1,1)에 투하되면 8방향으로 폭탄이 터집니다.

따라서

#	#	#	-	-
#	-	#	-	-
#	#	#	-	-
-	-	-	-	-

그리고 다시 (3,3)에 투하 되면 8방향으로 폭탄이 터집니다.

따라서

#	#	#	-	-
#	-	#	-	-
#	#	#	#	#
-	-	#	-	#

폭탄 2개가 투하 되었을때 그 현장을 출력 해주세요.

(Direct 기법을 사용해서 코딩해주세요)

입력 예제

1 1

3 3

출력 결과

_ _

_ # _ _

#

_ _ # _ #



문제 9번

입력 →

image			
1	5	5	4
4	2	1	1
3	9	3	2
4	5	9	1

4×4 image배열이 있습니다.

먼저 **4x4 image배열에 숫자를 입력** 받아주세요.

image 배열의 특정 좌표를 지목하면,

2×3 사이즈의 합을 return 해주는 **rectSum 함수**를 만들어 주세요

예를들어 rectSum(0,0)을 호출하면 $1+5+5+4+2+1 = 18$ 이 return 됩니다.

위 예제에서는 (2,0)이 최대 합 입니다.

이 함수를 활용하여 2x3 사이즈의 합의 **최대값이 나오는 좌표**를 찾아주세요.

(direct를 쓰는 문제가 아닙니다)

입력 예제

1 5 5 4

4 2 1 1

3 9 3 2

4 5 9 1

출력 결과

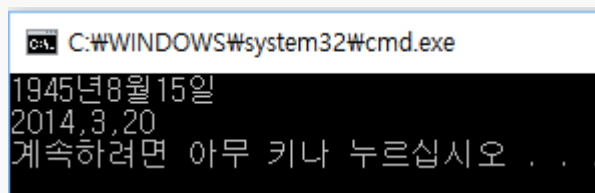
(2,0)



날짜를 다루는 Date 클래스를 작성하고자한다. Date를 이용하는 main()과 실행 결과는 다음과 같다. 클래스 Date를 작성하여 아래 프로그램에 추가하라.

```
int main()
{
    Date birth(2014, 3, 20);
    Date independenceDay("1945/8/15");
    independenceDay.show();
    cout << birth.getYear() << ',' << birth.getMonth() << ',' << b
    irth.getDay() << endl;

    return 0;
}
```



C:\WINDOWS\system32\cmd.exe
1945년8월15일
2014,3,20
계속하려면 아무 키나 누르십시오 . . .

복습문제



문제 1번



※ main 함수에서 a, b를 출력 해주세요.

입력 예제

4 2 5 3 8

출력 결과

a=8

b=2



문제 2번

5×4 배열에 0 또는 1로 구성된 숫자들을 입력받아주세요.

입력받은 배열에 있는 숫자 1은 세포를 뜻합니다.

세포 1주변에 아무것도 없으면 세포들이 안정된 상태입니다.

즉, 8방향 모두 아무런 숫자가 없어야 합니다.

아래 예제를 참고하여,

입력받은 세포상태가 "안정된 상태"인지 "불안정한 상태"인지 출력 하세요.

1			
			1
	1		
			1

← 이 상태는 숫자 1 주변에 아무 숫자가 없기때문에 안정된 상태 입니다.

	1		1
	1		
			1

← 하지만, 이 상태는 숫자가 붙어 있기때문에 불안정한 상태입니다.

1			1
		1	
1			

← 안정된 상태인지 불안정한 상태인지 출력하는 프로그램을 작성해주세요.

출력결과 : 안정된 상태

입력 예제

0 0 0 0

1 0 0 0

0 0 1 0

0 0 0 0

1 0 0 1

출력 결과

안정된 상태



문제 3번

1~16 사이의 숫자 4개를 입력 받고 배열에 채워주세요.

총 16칸짜리인 **4x4 배열**을 만들어 주세요.

4x4 배열에 아래와 같이 번호를 매긴다고 했을때

입력받은 숫자 4개에 해당하는 번호에 **값을 1부터 순차적으로 채워준 후 출력**해주세요.

아래 예제를 참고하여 소스코드를 작성해주세요.

입력: 3 10 11 15

3	10	11	15
---	----	----	----

4x4 배열

1	2	3	4
5	6	7	8
9	10	11	12
13	14	15	16



1부터 순차적으로 값 채우기

		1	
	2	3	
		4	

출력: 0 0 1 0
0 0 0 0
0 2 3 0
0 0 4 0

입력 예제

3 10 11 15

출력 결과

0 0 1 0

0 0 0 0

0 2 3 0

0 0 4 0



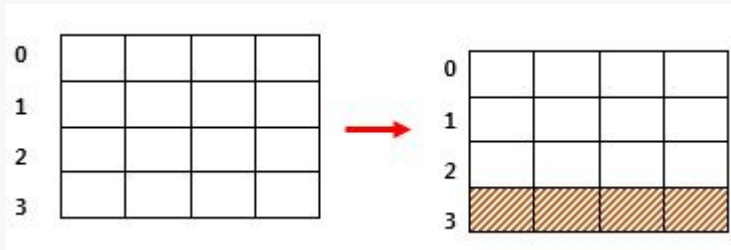
문제 4번

가로 또는 세로를 색칠하는 프로그램을 짜야 합니다.

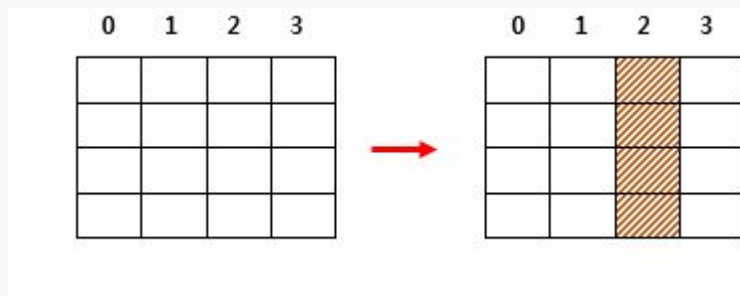
4×4 배열 하나를 준비 해 주세요.

만약

G 3이라고 입력 받으면 **가로 3번줄**이 색칠 됩니다.



S 2를 입력 받으면 **세로 2번줄**이 색칠 됩니다.



이러한 규칙으로 **3개의 명령어를 입력 받고 결과를 출력** 하세요.

ex)

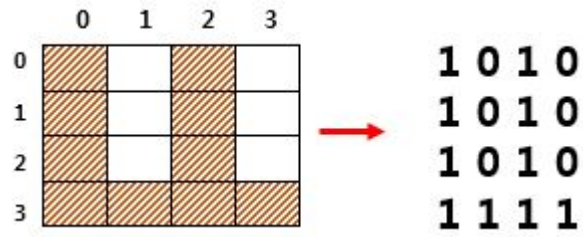
입력:

G 3

S 2

S 0

출력:



입력 예제

G 3

S 2

S 0

출력 결과

1 0 1 0

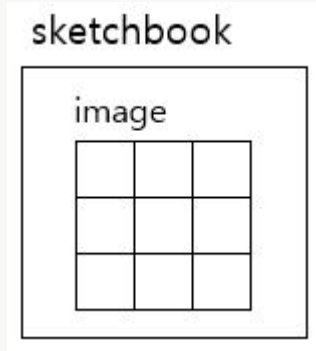
1 0 1 0

1 0 1 0

1 1 1 1



문제 5번



왼쪽 그림과 같은

sketchbook 구조체를 정의

하고,

구조체 변수 1개

를 만들어 주세요.

이미지에 들어갈 내용을 **구조체 변수인 image char배열에 입력받고,**

어떤 무늬로 구성되어 있는지 **알파벳 순서대로 출력**하세요.

(입력되는 문자는 모두 대문자입니다)

ex)

[입력] [출력]

ATK AGKT

AAA

TTG

입력 예제

BBB

BCD

DZZ

출력 결과

BCDZ



Level19.5 비밀 위치 찾기 [난이도 : 4]

문제 6번 [숙제 목록보기]

map				pattern	
A	B	G	K	A	B
T	T	A	B	C	D
A	C	C	D		
↑ 하드코딩				↑ 입력	

3×4 **map**배열은 위와 같이 **하드코딩** 하고,

2×2 **pattern**배열을 **입력** 받아주세요.

그리고 map배열에 **pattern**라는 패턴 배열이 **존재하는지 확인**하고
몇개인지 출력 하면 됩니다.

pattern을 체크 하는 함수를 이용하여 구현해주세요.

만약, pattern이 존재하고 1개가 발견되었다면 "**발견(1개)**" 출력

만약, pattern이 없다면 "**미발견**" 출력

입력 예제

AB

CD

출력 결과

발견(1개)



문제 6번

map				pattern	
A	B	G	K	A	B
T	T	A	B	C	D
A	C	C	D		
↑ 하드코딩				↑ 입력	

3×4 **map**배열은 위와 같이 **하드코딩** 하고,

2×2 **pattern**배열을 **입력** 받아주세요.

그리고 map배열에 **pattern**라는 패턴 배열이 **존재하는지 확인**하고
몇개인지 출력 하면 됩니다.

꼭 함수를 이용하여 구현해주세요.

만약, pattern이 존재하고 1개가 발견되었다면 "**발견(1개)**" 출력

만약, pattern이 없다면 "**미발견**" 출력

입력 예제

AB

CD

출력 결과

발견(1개)



문제 7번

map			bitarray	
3	5	1		
3	8	1		
1	1	5		

2×2 size의 **bitarray** 배열을 입력 받고, **map** 배열을 하드코딩 하세요.

bitarray를 map의 (0,0)에 masking하면 나오는 값은

3	5
3	

이고 합은 11입니다.

bitarray를 map에다 masking 후

합을 구해주세요.

합을 구하는 기능은 함수로 구현해주세요.

해당 함수를 이용해 가장 큰 값이 나오는 좌표를 출력 하세요.

입력 예제

1 1

1 0

출력 결과

(0,1)



문제 8번

은행에서 사용하는 프로그램을 작성하기 위해, 은행 계좌 하나를 표현하는 클래스 Account가 필요하다. 계좌 정보는 계좌의 주인, 계좌 번호, 잔액을 나타내는 3 멤버 변수로 이루어진다. main() 함수의 실행 결과가 다음과 같도록 Account 클래스를 작성하라.

```
int main()
{
    Account a("kitae", 1, 5000);

    a.deposit(50000);
    cout << a.getOwner() << "의 잔액은 " << a.inquiry() << endl;
    int money = a.withdraw(20000);
    cout << a.getOwner() << "의 잔액은 " << a.inquiry() << endl;

    return 0;
}
```



문제 9번

CoffeeMachine 클래스를 만들어보자.

main() 함수와 실행 결과가 다음과 같도록 CoffeeMachine 클래스를 작성하라.

에스프레소 한 잔에는 커피와 물이 각각 1씩 소비되고, 아메리카노의 경우 커피는 1, 물은 2가 소비되고, 설탕 커피는 커피 1, 물 2, 설탕 1이 소비된다.

CoffeeMachine 클래스에는 어떤 멤버 변수와 어떤 멤버 함수가 필요한지 잘 판단하여 작성하라.

```
int main()
{
    CoffeeMachine java(5, 10, 3);
    java.drinkEspresso();
    java.show();
    java.drinkAmericano();
    java.show();
    java.drinkSugerCoffee();
    java.show();
    java.fill();
    java.show();

    return 0;
}
```